ZEE COLLECTION

Corrected Master Blueprint — Draft for Owner Review

Review date: 25 October 2026 • Implementation paused • No resources changed

Authorization boundary: Planning artifact only. No code, database resources, DNS, Cloudflare settings, or deployments are changed by this document.

Document status: Draft for owner review — **NOT approved for implementation**

Review date: 25 October 2026

Business: ZEE COLLECTION

Domain: `zeecollection.tech` (ownership, DNS, and current deployment state are unverified)

Primary workflow: Android + SPCK Editor + GitLab web interface + Cloudflare dashboard

Required stack: HTML, CSS, vanilla JavaScript; Cloudflare Pages; Cloudflare Pages Functions; Cloudflare D1; Cloudflare R2

Research note: Time-sensitive official-source checks below were performed on 25 October 2026. Recheck before implementation/launch.

Authorization boundary: This is a planning artifact only. No application files, database resources, buckets, bindings, DNS, secrets, repository settings, integrations, or deployments are created or changed by this document. The previous pause remains in force. Do not implement until the owner has reviewed this blueprint and explicitly authorizes the next step.

1. Executive summary

Build a maintainable, mobile-first premium e-commerce storefront for ZEE COLLECTION. The initial launch catalogue is **watches**. Rings and glasses are possible later categories, not launch scope unless the owner changes this decision. The store should be trustworthy, visually restrained, accessible, fast on mobile, and honest about products, price, stock, delivery, payment, returns, and support.

The frontend is static HTML/CSS/vanilla JavaScript. Dynamic product/order/admin operations run through Cloudflare Pages Functions. D1 is the source of truth for structured records; R2 stores product-image objects. Sensitive data and private payment evidence must not be publicly served. GitLab is the intended source repository; Android/SPCK is the primary editing workflow.

The project is delivered in phases with written acceptance criteria, evidence of tests actually run, a status report, and an explicit owner approval gate after each phase. A visually complete page is not evidence of a functioning or secure store.

Confirmed constraints

- Cloudflare Pages + Pages Functions + D1 + R2 only for hosting, backend, database, and object storage.
- HTML, CSS, and vanilla JavaScript only unless the owner explicitly approves a different frontend stack.
- Android and SPCK Editor are the primary editing environment. Do not silently require a desktop, local terminal, Node/npm, or unavailable mobile workflow.
- Product information and stock persist in D1; product images persist in R2. Do not use `localStorage`, in-memory browser state, or `products.json` as authoritative persistence.
- Do not assume the domain, repository, Cloudflare project, database, bucket, deployment, or integration already exists. Inspect/verify before any action.
- Do not request passwords, API keys, recovery codes, or other secrets in chat.
- No deletion, overwriting, migration, DNS change, binding change, or deployment without explicit authorization and a rollback plan.
- Every code change must be delivered as a downloadable complete file; when practical, provide a complete project ZIP. Every substantive response must also have a downloadable PDF copy, including code where applicable.

Recommended defaults (not yet owner-approved)

- Launch with watches only.
- Start with guest checkout plus a protected guest order lookup; add customer accounts only if there is a clear business need.
- Start with Cash on Delivery (COD) only if a courier and seller eligibility are confirmed. Add one advance-payment method only after merchant eligibility, fees, reconciliation, and refund handling are verified.
- Use manual courier dispatch/tracking if an official API is unavailable or not worth the cost.

- Use a click-to-chat WhatsApp link rather than automated messaging initially.
- Keep payment evidence private and make an authorized human verify it; screenshots alone never trigger automatic payment approval.
- Do not add analytics/ad pixels until privacy, consent, and business value are decided.

2. Comparison against original source documents

2.1 Original sources considered

1. **ZEE COLLECTION — Master Blueprint Specification** (8 pages).
2. **ZEE COLLECTION — Professional Master Project Plan** (10 pages).
3. **ZEE COLLECTION — Blueprint Amendment v2: Legal, CRO, and Style** (legal/customer trust/copy rules).

2.2 Missing or under-specified in the earlier consolidated blueprint

The earlier consolidated document was directionally useful but did not fully operationalize several source requirements. This corrected version adds or strengthens:

- A requirements classification separating confirmed constraints, recommendations, assumptions, legal obligations requiring verification, and unresolved owner decisions.
- A more explicit API inventory, error conventions, authorization matrix, and order/payment/shipping state transitions.
- A proposed D1 schema with relationships, data-minimization notes, unique constraints, idempotency, audit records, retention, and migration/rollback expectations.
- A threat model for admin compromise, customer/order enumeration, guest lookup tokens, upload abuse, replayed payment evidence, stock races, price tampering, and abuse/rate limits.
- Backup and recovery planning, including the distinction between provider point-in-time recovery and an independently tested export/restore plan.
- Mobile accessibility requirements, keyboard and screen-reader behavior, contrast, focus states, reduced-motion support, and form errors.
- Product SEO, metadata, canonical URLs, structured data only where truthful, crawl/index controls for private/admin/order pages, image optimization, and performance checks.
- More complete legal/customer policy content: seller identity, complaint route, privacy notice, delivery and return/refund terms, invoices/receipts, data retention, and no unsupported claims.
- Clearer tax modelling: statutory tax, withholding, delivery fee, courier cost, gateway fee, and store-charged fee must not be conflated.
- Operational failure paths: payment provider unavailable, duplicate submissions, stock exhausted during checkout, failed uploads, orphaned images, courier API unavailable, quota exhaustion, and failed deployments.
- A documented end-of-phase report and test evidence standard.

2.3 Potential contradictions or ambiguities reconciled

- **Business category:** One prompt describes a “premium shoes and fashion” store, while the project context and earlier discovery specify a watch-first launch with rings/glasses later. This blueprint treats **watches as launch scope** and shoes/fashion as a broader brand description only. Owner should confirm.
- **Domain ownership:** `zeecollection.tech` is named, but ownership/current DNS are not proven. Do not purchase it again or change nameservers before checking registrar and Cloudflare state.
- **“Fresh start” vs “do not overwrite”:** The user previously wanted a fresh project, but later explicitly instructed not to delete or overwrite existing work. The latest preservation instruction governs. Inspect before changing anything.
- **START PROJECT authorization:** The source specification required the exact phrase `START PROJECT` to begin the approved implementation workflow. A later, more recent instruction paused implementation pending review. Therefore, the earlier phrase does not override the pause. After reviewing this corrected blueprint, the owner must explicitly authorize the next step; no implementation occurs merely because this document exists.
- **Phase completion:** The source documents require stopping after each phase. This blueprint keeps that requirement: the next phase begins only after the owner confirms the prior phase is complete.
- **Pages vs Workers:** Cloudflare currently recommends Workers for some new applications, but the owner’s explicit architecture constraint is Pages + Pages Functions + D1 + R2. This remains unchanged unless the owner expressly approves a migration.
- **Static catalogue vs persisted catalogue:** Static HTML/CSS/JS is acceptable for the shell, but published product data must come from D1-backed API responses. A static `products.json` cannot be the production source of truth.
- **Manual payment evidence:** A receipt/screenshot can be submitted as evidence, but it is not proof of settlement and must not automatically approve a payment.
- **Legal “compliance” claims:** The project must not claim it is “NCCIA compliant”, “FIA approved”, “100% legally protected”, or otherwise certified without actual competent-authority authorization and legal review.

2.4 Requirements retained as unnecessary or unsupported until verified

- Any fixed **4% tax** or other hard-coded tax rate: **not accepted**. Tax and withholding depend on applicable law and seller circumstances.
- Any assumed courier name, coverage, delivery window, fee, COD settlement period, or API access: **not accepted until verified**.
- Any assumed Raast/payment-provider API access, merchant eligibility, fee, settlement, or refund flow: **not accepted until the provider/bank confirms it**.
- Any unconditional return/refund/warranty guarantee: **not accepted until the owner approves a policy that can actually be honored and legal requirements are checked**.
- Fake reviews, fabricated ratings, false scarcity, invented customer counts, unsupported “best/official/certified” claims, and copied competitor text/design: **prohibited**.
- A large legal footer listing laws to look official: unnecessary. Publish concise accurate customer policies; keep the research/legal register internally.
- A claim that a privacy bill remains unenacted based only on an old draft listing: unsupported. The official MOITT page lists draft versions, but that fact alone does not establish the complete current legislative position.
- A claim that the business is legally compliant: unsupported without facts about seller identity, location, registration, goods, tax status, policies, and professional review.
- External hosting, databases, object storage, frontend frameworks, or third-party persistence services: prohibited by the current stack constraint unless explicitly approved.

3. Requirements and decision register

Use these labels throughout implementation documentation:

- **CONFIRMED:** explicitly required by the owner or consistently established in source materials.
- **RECOMMENDATION:** a proposed safer or more maintainable approach, not yet an owner decision.
- **ASSUMPTION:** used provisionally because a necessary fact is missing.
- **LEGAL CHECK REQUIRED:** the relevant law or official source must be checked against the actual seller and transaction.
- **UNRESOLVED:** owner decision or external provider confirmation is needed.
- **VERIFIED FACT (dated):** supported by a current official source checked on the stated date.

Topic	Classification	Current position
Frontend	CONFIRMED	HTML, CSS, vanilla JS only unless explicitly approved otherwise
Hosting/backend/data/storage	CONFIRMED	Cloudflare Pages, Pages Functions, D1, R2 only
Editing	CONFIRMED	Android + SPCK Editor primary workflow
Repository	CONFIRMED INTENT	GitLab planned; actual repository existence/state unverified
Launch catalogue	PROVISIONAL	Watches first; rings/glasses later
Currency	ASSUMPTION	PKR likely, but owner must confirm
Sales area	UNRESOLVED	Pakistan-wide, selected regions, or other markets
Seller legal identity	UNRESOLVED	Individual/sole proprietor/company and required disclosures
Dispatch origin	UNRESOLVED	Needed for courier, delivery, tax, and consumer-law checks
Domain/DNS	UNVERIFIED	Must inspect before purchase or DNS edits
Payment methods	UNRESOLVED	Merchant eligibility and terms not confirmed
Courier	UNRESOLVED	No provider selected or verified
Customer accounts	RECOMMENDATION	Guest checkout first unless accounts have clear value
WhatsApp	RECOMMENDATION	Click-to-chat first; automated messaging deferred
Analytics/marketing pixels	DEFERRED	No third-party tracking until privacy/value review
Legal/tax compliance	NOT ESTABLISHED	Needs seller-specific legal/tax review

4. Architecture

4.1 Components

1. **Cloudflare Pages:** serves static HTML, CSS, JavaScript, product SEO pages/shell, and static assets.
2. **Cloudflare Pages Functions:** API routes; authorization; input validation; D1 queries/transactions; R2 upload/download authorization; rate limits; server-side calculations.
3. **Cloudflare D1:** authoritative structured store data, order records, stock, user/admin records if enabled, payment events, settings, and audit logs.

4. **Cloudflare R2:** product-image objects. Product images may be public only through an intentional public delivery path. Payment evidence and other sensitive files must remain private.
5. **GitLab:** source control and review history. Never commit secrets, production customer data, or private payment evidence.
6. **SPCK Editor on Android:** primary editing workflow. Any step requiring a desktop/terminal must be identified before it is prescribed, with a mobile-compatible alternative or a clear blocker.

No external runtime, database, object storage, hosting, or frontend framework is introduced.

4.2 Data flow

- Browser requests public catalogue/product information from Pages Functions.
- Function validates query parameters and returns only published/public fields from D1.
- Browser submits an order intent, not trusted prices or calculated totals.
- Function re-reads product status, current price, variant, stock, delivery rules, and applicable tax configuration from D1; calculates totals server-side; and creates the order using a transaction/conditional stock operation.
- Payment provider/manual workflow updates payment status only through an authorized, traceable path.
- Admin actions require server-side authorization on every request; hiding admin buttons in the frontend is not security.
- Product upload is authorized by a Function; image bytes go to R2 through a controlled path; D1 stores object metadata and relationship.
- Private order/payment records are never exposed via public R2 URLs or unguessable-but-public file paths.

4.3 Static frontend boundaries

- Use semantic HTML, CSS design tokens, and vanilla JS modules/files as appropriate.
- No frontend secret, database credential, signing key, admin password, or private API key.
- No authoritative product/order persistence in browser storage.
- Cart contents may be held temporarily in memory or non-sensitive browser storage if later approved, but server totals and availability always override client values. Do not store credentials, private payment evidence, or authoritative stock/pricing in browser storage.
- Offline UI may show a shell or previously loaded non-sensitive content, but must not claim an order/payment succeeded while offline.
- Do not add a service worker/offline cache until cache rules ensure private pages, account/order data, admin pages, and payment flows are not cached unsafely.

4.4 Proposed file layout (not created)

```
/
  index.html
  shop.html
  product.html
  cart.html
  checkout.html
  order-status.html
  contact.html
  policies/
    privacy.html
    delivery.html
    returns-refunds.html
    terms.html
  admin/
    index.html
    products.html
    orders.html
    settings.html
  assets/
    css/
      design-tokens.css
      base.css
      components.css
      pages.css
    js/
      api.js
      storefront.js
      product.js
      cart.js
      checkout.js
      order-status.js
      admin-auth.js
      admin-products.js
      admin-orders.js
    images/
  functions/
    api/
      health.js
      products.js
      products/[slug].js
      admin/auth/login.js
      admin/auth/logout.js
      admin/auth/session.js
```

```
      admin/products/index.js
      admin/products/[id].js
      admin/uploads/sign.js
      orders/index.js
      orders/lookup.js
      orders/[id].js
      payments/submit-evidence.js
      admin/payments/[id]/verify.js
      admin/orders/[id]/status.js
      admin/settings.js
  migrations/
    0001_initial_schema.sql
  docs/
    AI_CONTEXT.md
    BLUEPRINT.md
    CHANGELOG.md
    TESTING.md
```

This is a proposal only. The final route/file layout should be adapted to verified Pages Functions routing conventions before implementation. Do not create these files during the paused review.

5. Proposed D1 schema

The schema below is a logical proposal, not a migration. Final column types, indexes, retention, constraints, and state rules must be reviewed before any database creation.

5.1 Tables

`admin_users`

- `id` (primary key)
- `username_normalized` (unique)
- `password_hash` (modern password hash; never plaintext)
- `role` (initially a narrowly scoped admin role; do not rely on frontend role flags)
- `is_active`
- `created_at`, `updated_at`, `last_login_at`
- Optional credential-rotation/recovery metadata that does not expose secrets

Security: Use a modern password hashing library compatible with the Cloudflare runtime. Never roll a custom password hash. Use MFA if a feasible, secure implementation is confirmed; otherwise strengthen login rate limits and account recovery. Do not make account recovery depend only on an easily guessed username.

`admin_sessions`

- `id` (primary key)
- `admin_user_id` (foreign key)
- `session_token_hash` (store a hash of a high-entropy opaque token, not the raw token)
- `created_at`, `expires_at`, `revoked_at`, `last_seen_at`
- Optional security metadata minimized to what is operationally justified

Use Secure, HttpOnly, SameSite cookies for browser sessions where appropriate. Protect state-changing requests against CSRF. Do not put session tokens in localStorage or URLs. Session lifetime, rotation, logout, and revocation must be explicit.

`customers` (only if accounts are enabled)

- `id`
- `email_normalized` and/or `phone_normalized` (unique only where the business rules require)
- `password_hash` if password accounts are enabled
- `display_name`
- `is_active`, `created_at`, `updated_at`
- Minimal consent/communication preferences where needed

Do not collect date of birth, identity documents, or unrelated personal data without a documented need.

`products`

- `id`
- `slug` (unique)

- `name`
- `short_description`, `description`
- `category_id`
- `status` (`draft`, `published`, `archived`)
- `brand` if applicable and truthful
- `seo_title`, `seo_description`
- `created_at`, `updated_at`, `published_at`
- Optional `deleted_at` for a controlled soft-delete workflow

Public queries must filter `status='published'` and omit internal fields.

`categories`

- `id`, `slug` (unique), `name`, `description`, `sort_order`, `is_active`
- `seo_title`, `seo_description`

`product_variants`

- `id`, `product_id` (foreign key)
- `sku` (unique where present)
- `variant_name` / structured option values (for example, strap/color)
- `price_minor` integer
- `currency_code`
- `stock_quantity` integer, non-negative
- `is_active`
- `created_at`, `updated_at`

Use integer minor units for money (e.g. paisa where relevant), not floating-point calculations. Confirm currency and rounding rules before implementation. For products without variants, either use a single default variant or a clearly defined product-level inventory model; do not duplicate stock in two places.

`product_images`

- `id`, `product_id` (foreign key)
- `variant_id` nullable foreign key if variant-specific images are needed
- `r2_object_key` (unique)
- `alt_text`
- `sort_order`, `is_primary`
- `mime_type`, `byte_size`, `width`, `height`
- `created_at`

Do not store public/private R2 credentials or make object keys alone a security control. Product-image public delivery is a deliberate publication decision.

`orders`

- `id` (opaque public order identifier; not a sequential guessable ID)
- `order_number` (human-friendly unique reference, not an authorization secret)
- `customer_id` nullable
- `guest_lookup_token_hash` nullable (hash only; token shown to guest once)
- `status` (documented order lifecycle)
- `payment_status` (separate lifecycle)
- `shipping_status` (separate lifecycle)
- `currency_code`
- `subtotal_minor`
- `discount_minor`
- `delivery_fee_minor`
- `tax_minor` only when applicable and professionally verified
- `withholding_note`/accounting references where necessary; do not automatically treat withholding as checkout tax

- `total_minor`
- Snapshot of required recipient/contact/delivery address fields
- `customer_note` with length limits
- `idempotency_key` or equivalent duplicate-submission protection
- `created_at`, `updated_at`, `confirmed_at`, `cancelled_at`

Store only the address/contact details needed for fulfilment and legal obligations. Define who can read them and when they are deleted or anonymized.

`order_items`

- `id`, `order_id` (foreign key)
- `product_id` nullable reference for historical linkage
- `variant_id` nullable reference
- Snapshot: `product_name`, `sku`, `variant_description`
- `unit_price_minor`, `quantity`, `line_total_minor`
- Must preserve what the customer ordered even if the product is later edited or archived.

`payment_attempts`

- `id`, `order_id`
- `method` (COD, manual transfer, provider identifier)
- `status` (`pending`, `evidence_submitted`, `under_review`, `paid`, `failed`, `refunded`, `cancelled` as appropriate)
- `amount_minor`, `currency_code`
- Provider transaction/reference ID where applicable
- `idempotency_key` / unique provider event ID
- `submitted_at`, `verified_at`, `verified_by_admin_id`
- Failure/refund metadata with no secrets

Never mark paid based only on a browser redirect or screenshot. Prefer verified server-to-server provider events where available; otherwise require an authorized human to confirm the transaction independently.

`payment_evidence`

- `id`, `payment_attempt_id`
- `r2_object_key` (private bucket/object)
- `mime_type`, `byte_size`, `submitted_at`
- `review_status`, `reviewed_by_admin_id`, `reviewed_at`
- Optional reason codes; avoid unnecessary free-text sensitive details

Evidence must be private and retrieved only through an authenticated, authorized Function that checks the reviewer's role. Define deletion/retention.

`shipments`

- `id`, `order_id`
- `courier_name` only after selection
- `tracking_reference`
- `status`, `dispatched_at`, `delivered_at`, `returned_at`
- Optional API sync metadata
- Do not claim courier API integration until tested with a real eligible account.

`returns_refunds`

- `id`, `order_id`, optional `order_item_id`
- `type` (`return`, `exchange`, `refund`, `replacement`)
- `status`, `reason_code`, `customer_note`
- `requested_at`, `decided_at`, `completed_at`
- `refund_amount_minor`, `currency_code`
- `handled_by_admin_id`

`store_settings`

- `key` (primary key), `value_json`, `updated_at`, `updated_by_admin_id`
- Allow only a strict allowlist of settings that admins may change. Never store raw secrets in general settings JSON.

`audit_logs`

- `id`, `actor_admin_user_id` nullable for system actions
- `action`, `entity_type`, `entity_id`
- `result`, `created_at`, `request_id`
- Minimal metadata needed for investigations
- Never log passwords, raw session tokens, payment credentials, full card data, or unnecessary personal data.

`rate_limit_events` (or equivalent persistent rate-limit structure)

- Keyed by a privacy-conscious hashed subject/route/time window, with expiry/cleanup strategy.
- Must be effective across requests/instances; do not rely solely on in-memory counters.
- Consider Cloudflare platform protections where available, but verify product/plan applicability. D1-based rate limits can create contention under heavy abuse; if so, pause and request approval before introducing another Cloudflare service.

Optional supporting tables

- `order_status_events` and `payment_status_events` for an immutable transition history.
- `inventory_movements` to record adjustments, sale reservations, releases, and admin stock changes.
- `complaints` if complaint handling needs a separate workflow.

5.2 Relationships and integrity rules

- One product has many variants and images.
- One customer may have many orders; guest orders have no customer foreign key.
- One order has one or more order items, one or more payment attempts, and zero or more shipments/return records.
- All money is integer minor units with explicit currency.
- Use foreign keys where supported and appropriate, unique constraints for slugs/SKUs/provider event IDs, and indexes for common lookups.
- Never trust browser-provided product price, line total, delivery charge, tax, discount eligibility, stock, role, payment state, or order status.
- Stock must not become negative under concurrent checkout requests. Use a transaction and/or conditional update that succeeds only when sufficient stock remains.
- Make order creation idempotent to prevent duplicate orders from double taps, retries, or network timeouts.
- Define how stock is reserved/released on pending payment, cancellation, timeout, failed payment, and COD order confirmation.
- Apply schema migrations as reviewed, versioned, reversible where practical, and tested on a non-production database first.

6. API plan

All routes are proposed; route layout and Cloudflare binding names must be verified before coding. JSON responses should use consistent errors and never expose stack traces or secrets.

6.1 Public routes

Method / route	Purpose	Controls
`GET /api/health`	Minimal health status	No secrets, no database internals
`GET /api/products`	Published catalogue, category/filter/sort/pagination	Validate inputs, cap page size, only published fields
`GET /api/products/:slug`	Published product details	Only published product and public images
`GET /api/categories`	Active public categories	No drafts/internal settings
`POST /api/orders`	Create guest/customer order	Server totals, validation, stock checks, idempotency, abuse limits
`POST /api/orders/lookup`	Guest order lookup exchange	High-entropy token or equivalent verification; generic errors to reduce enumeration
`GET /api/orders/:id`	Order status	Customer ownership/session or verified guest capability required
`POST /api/payments/evidence`	Submit payment evidence if approved	Order capability/ownership, private R2, strict upload limits, rate limits
`POST /api/contact/complaints`	Optional support/complaint intake	Data minimization, rate limits, clear receipt confirmation

6.2 Admin routes

| Method / route | Purpose | Controls |

| --- | --- | --- |

| `POST /api/admin/auth/login` | Admin login | Password hash verify, throttling, generic failure response |

| `POST /api/admin/auth/logout` | Revoke session | CSRF protection and cookie clearing |

| `GET /api/admin/auth/session` | Current session/role | Server-derived role; no client trust |

| `GET /api/admin/products` | Product management list | Admin authorization, pagination |

| `POST /api/admin/products` | Create draft product | Validation, audit record |

| `PATCH /api/admin/products/:id` | Edit product | Field allowlist, concurrency/version check if useful |

| `POST /api/admin/products/:id/publish` | Publish/unpublish | Required-field checks, audit log |

| `DELETE /api/admin/products/:id` | Archive/delete workflow | Prefer archive where historical orders reference product; audit |

| `POST /api/admin/uploads` | Upload product image | Admin auth, content validation, R2 key generated server-side |

| `DELETE /api/admin/images/:id` | Remove image | Ensure not referenced as primary; cleanup/rollback rules |

| `GET /api/admin/orders` | Search/manage orders | Least privilege, pagination, minimize PII exposure |

| `PATCH /api/admin/orders/:id/status` | Allowed order transition | State machine validation and audit |

| `POST /api/admin/payments/:id/verify` | Verify/decline payment | Authorized admin, independent evidence, reason and audit |

| `POST /api/admin/refunds/:id` | Record/execute refund | Only after refund mechanism/policy verified; idempotency |

| `PATCH /api/admin/shipments/:id` | Dispatch/tracking updates | Valid transitions, audit |

| `GET/PATCH /api/admin/settings` | Read/change allowlisted store settings | Admin auth; no secret disclosure; audit |

Do not expose an admin registration endpoint publicly. Bootstrap the first admin through a controlled, documented, one-time procedure using secrets entered directly in the provider dashboard—not chat. Disable bootstrap after setup.

6.3 Common API behavior

- Use appropriate HTTP status codes (`400`, `401`, `403`, `404`, `409`, `413`, `422`, `429`, `500`, `503` as relevant).
- Return a stable error object such as `{ "error": { "code": "OUT_OF_STOCK", "message": "Availability changed. Please review your cart." } }`.
- Do not reveal whether an email/phone/order token exists when doing so enables enumeration.
- Attach a request/correlation ID to logs and user-safe error messages.
- Validate content type, body size, field lengths, allowed enum values, IDs, pagination, and uploaded content.
- Use parameterized SQL. Do not concatenate user input into SQL identifiers or clauses.
- Apply origin/CORS policy intentionally. Same-origin deployment is preferred. CORS is not authentication.
- Never return password hashes, session hashes, guest tokens, private R2 keys, raw error stacks, or full internal audit records to public clients.

7. Security architecture and threat model

7.1 Admin authentication

Risks: credential stuffing, brute force, session theft, CSRF, role tampering, insecure recovery, forgotten bootstrap accounts, admin URL treated as a secret.

Controls:

- Server-side password verification with a maintained password-hashing implementation supported in the Cloudflare runtime.
- Rate-limit login attempts persistently/effectively and use generic failure messages.
- Opaque, high-entropy sessions; store only token hashes where feasible; Secure + HttpOnly + SameSite cookies; expiry, rotation, revocation, logout.
- Protect cookie-authenticated state-changing requests against CSRF; validate origin/referer as defense-in-depth.
- Enforce role/permission checks on every admin API call, not just page loading.
- Least privilege, audited admin actions, secure bootstrap and recovery, and a documented emergency disable/recovery process.
- MFA is recommended if feasible with a verified, maintainable method; exact implementation remains a decision after capability review.
- No admin secrets in frontend, GitLab, screenshots, or chat.

7.2 Customer data and order ownership

Risks: IDOR (changing an order ID to see another order), accidental public APIs, excessive PII collection, log leakage, long retention.

Controls:

- Default-deny authorization for customer/order routes.
- Customers can access only their own records; check ownership server-side on every request.
- Guest order lookup uses a high-entropy random capability token shown once, with only a hash stored. The human-friendly order number is not a secret.
- Prefer exchanging the guest token for a short-lived secure session/capability rather than putting a long-lived token in a URL that may leak through browser history, referrers, screenshots, or logs.
- Generic errors and rate limits to prevent token/order enumeration.
- Minimize data, define retention/deletion/anonymization, and do not log full addresses or tokens.
- Use HTTPS and restrictive cache headers for private pages/API responses.

7.3 Payment verification

Risks: fake screenshots, duplicate/replayed callbacks, forged browser redirects, wrong amount/order association, unauthorized admin approval.

Controls:

- Separate order, payment, and shipping states.
- Payment provider callbacks/webhooks must be authenticated and signature-verified according to current provider documentation; enforce idempotency and event uniqueness.
- Verify exact order, amount, currency, and provider transaction reference.
- If using manual transfer, an authorized admin independently checks the bank/payment record. An uploaded screenshot is only a claim, not proof.
- Record who verified/declined, when, and why. Require appropriate permission for refunds.
- Never store card security codes or unnecessary payment credentials.
- No payment provider is considered selected or integrated until eligibility, documentation, fee schedule, refund process, and test results are verified.

7.4 Image and file uploads

Risks: malicious files, MIME spoofing, huge uploads, decompression bombs, executable content, path/key manipulation, public leakage, orphaned objects.

Controls:

- Admin authorization before upload/signing; server-generated object keys.
- Allow only approved image formats; check extension, declared MIME, file signature, size, and dimensions; decode/re-encode where a supported safe library/runtime path exists.
- Reject SVG unless a safe sanitization policy is explicitly implemented; avoid HTML/SVG execution risks.
- Limit size, dimensions, count, and upload rate.
- Product images are public only after explicit association/publication. Payment evidence remains private.
- Never accept an arbitrary R2 key from the client as authorization.
- Track uploads before database commit and clean up failed/orphaned objects with a safe retryable process.
- Test unauthorized read attempts and public-bucket settings. R2 buckets are private by default, but enabling public access makes the objects internet-accessible; verify the exact configuration before launch.

7.5 Price, tax, delivery, discounts, and inventory

Risks: client price tampering, negative stock, overselling, stale carts, tax misclassification, race conditions, duplicate order creation.

Controls:

- Browser sends product/variant IDs and quantities, not authoritative totals.
- Server re-reads price, publication state, stock, currency, delivery rule, discount eligibility, and verified tax configuration.
- Use integer minor units; define rounding and currency rules.
- Apply atomic conditional stock updates/transactions; test simultaneous requests for the final item.
- Idempotency keys prevent duplicate orders on retries/double taps.
- Record immutable order-item price/name snapshots.
- Keep government tax, income-tax withholding, payment fee, courier cost, and store delivery fee separate in data and UI.
- If tax treatment is unresolved, do not silently guess. Pause tax calculation/launch until qualified advice confirms how the seller must display and remit amounts.

7.6 Abuse, application, and operations security

- Validate all input and output-encode content; use safe DOM APIs rather than inserting untrusted product/customer content as raw HTML.
- Rate-limit login, order creation, guest lookup, evidence uploads, and contact forms.
- Use security headers appropriate to the actual site, including a tested Content Security Policy, `X-Content-Type-Options`, `Referrer-Policy`, and frame restrictions; do not copy a CSP that breaks the app without testing.
- Keep errors user-safe and logs private; log security-relevant events without secrets/needless PII.
- Keep dependencies minimal; no framework dependency unless approved. Review any added package for maintenance, bundle size, compatibility, and license.
- Test CORS, CSRF, access control, SQL injection, XSS, upload controls, enumeration, replay, concurrency, and quota exhaustion.
- Keep production and preview/test data separated. Never use real customer/payment data for routine testing.
- Establish backup/restore and incident-response steps before launch.

8. Order, payment, shipping, and returns workflows

8.1 Order workflow (proposed)

`draft cart → server validation → pending confirmation/payment as applicable → confirmed → processing → dispatched → delivered`

Exceptional states: `cancelled`, `failed`, `returned`, `partially returned`, `refunded` as needed. Do not allow arbitrary status changes; implement a documented transition table and audit each transition.

8.2 Payment workflow (separate)

Examples: `not_required/COD_due`, `pending`, `evidence_submitted`, `under_review`, `paid`, `failed`, `cancelled`, `refund_pending`, `partially_refunded`, `refunded`.

The final allowed values depend on the chosen payment methods. COD is not “paid” merely because an order is accepted; payment collection on delivery and remittance from courier may need separate operational/accounting records.

8.3 Shipping workflow (separate)

Examples: `not_dispatched`, `ready_to_dispatch`, `dispatched`, `in_transit`, `delivered`, `delivery_failed`, `return_to_origin`, `returned`.

Manual tracking is acceptable initially if no verified courier API is available. Never fabricate a tracking number or claim live tracking when the courier has not supplied it.

8.4 Failure paths that must be designed and tested

- Product becomes unpublished or out of stock after the cart was opened.
- Price changes between cart and checkout.
- Two users attempt to buy the final unit simultaneously.
- Customer double taps submit or retries after a timeout.
- D1 write fails after an R2 upload, or upload succeeds but D1 commit fails.
- Provider callback is duplicated, late, invalid, or out of order.
- Admin rejects evidence or mistakenly marks the wrong order paid.
- Courier API is unavailable or tracking is missing.
- A user attempts to view another order or a private evidence file.
- Daily Functions/D1 quotas are reached.
- Network goes offline during checkout.
- Deployment introduces a regression and must be rolled back.

9. Privacy, policies, legal, and tax review

This is technical planning, not legal or tax advice. It does not establish that the business is compliant. Actual obligations depend on seller identity/legal form, business and dispatch location, product type, customer markets, registration/tax status, transaction method, and current law. Obtain qualified Pakistani legal/tax advice where material.

9.1 Binding law vs policy/draft/recommendation

- **Enacted law:** a statute/ordinance enacted through the applicable process. Applicability to this specific store still needs analysis.
- **Rules/regulations/SROs:** may be binding within their scope; check the current text, effective date, amendments, and applicability.
- **Government policy:** policy direction/guidance is not automatically equivalent to binding legislation.
- **Draft bill:** proposed text; do not describe it as enacted law.
- **Engineering recommendation:** a sensible risk control, not by itself a statutory obligation.
- **Unresolved legal question:** must be marked as unresolved, not filled with a guess.

9.2 Official-source findings checked 25 October 2026

Electronic Transactions Ordinance, 2002 — enacted law. Pakistan Code hosts the Ordinance. It is relevant to electronic records and transactions, but the legal effect of a particular checkbox, click-wrap, message, or record should not be asserted without legal review. Keep order confirmations, consent where required, receipts/invoices, payment decisions, and status changes traceable and protected. Official source: https://pakistancode.gov.pk/pdffiles/administratordbc98dd49f2df3b1d07bb986dcceb9a3.pdf

Prevention of Electronic Crimes (Amendment) Act, 2025 — enacted source identified. The National Assembly published the amendment Act, and NCCIA has a laws listing. The site should use reasonable security, access control, logging, and abuse prevention, but those measures do not certify legal compliance. Do not use “NCCIA compliant” or “FIA approved” badges without genuine authorization. Official sources: https://na.gov.pk/uploads/documents/679b243193585_457.pdf and https://www.nccia.gov.pk/laws.php

Consumer protection — seller/location-specific verification still required. Consumer protection may depend on the province and markets served. The prior review did not establish one definitive, consolidated official source that resolves every applicable Punjab rule, process, disclosure, and remedy. Verify current Punjab and federal law using official texts and a qualified lawyer. Do not invent section numbers, complaint deadlines, return periods, or mandatory wording.

E-commerce Policy 2019–24 — policy, not a substitute for law. Ministry of Commerce policy materials can inform planning but should not be presented as a standalone binding compliance checklist. Official source: https://www.commerce.gov.pk/ecommerce-policy-2019/

E-commerce tax/withholding — enacted provisions and FBR guidance identified; seller-specific treatment unresolved. FBR Circular No. 01 of 2025–26 describes a section 6A e-commerce regime and withholding at 1% for digital payment through a payment intermediary and 2% for COD courier remittance under the described regime. The current Income Tax Ordinance version found was amended 20 February 2026. These rates are **not universal checkout sales-tax rates**. Sales tax is a separate question and may depend on goods, registration, province, statutory schedules, and seller circumstances. Do not hardcode 4% or label courier/gateway fees as tax. Confirm the seller’s registration, applicable income-tax treatment, withholding/final-tax implications, sales tax, invoice disclosures, return filing, and records with an FBR-qualified tax practitioner.

Official sources:

- FBR Income Tax Circular No. 01 of 2025–26: https://download1.fbr.gov.pk/Docs/2025841183918948CircularNo01of2025-26IncomeTax.pdf
- FBR Sales Tax/Federal Excise Circular No. 02 of 2025–26: https://download1.fbr.gov.pk/Docs/202584118361586CircularNO02of2025-26SalesTax%26FederalExcise.pdf
- FBR SRO 1429(I)/2025: https://download1.fbr.gov.pk/SROs/2025858833282SRO1429DATED04-08-2025.pdf
- Income Tax Ordinance amended 20 February 2026: https://download1.fbr.gov.pk/Docs/2026226162211364IncomeTaxOrdinance2001-Amended-20.02.2026.pdf

Personal data protection bill status — do not overstate. MOITT’s official legislation listing shows Personal Data Protection Bill versions as drafts, including a May 2023 draft. This confirms the status shown on that page, not that no later law or relevant rule exists anywhere as of the review date. Recheck the current Gazette/official legislation before launch. Privacy duties may also arise under other laws, contracts, provider terms, and consumer protection. Official source: https://www.moitt.gov.pk/Legislations

9.3 Required customer-facing policy pages (content requires owner/legal approval)

- **Seller identity/contact:** truthful business/trading name and practical support contact; disclose legal name, address, registration details, or other information when required for the actual business.
- **Privacy notice:** what is collected, why, sharing, retention/deletion, security practices, customer rights/contact route, and hosting/storage disclosures where required.
- **Terms/order conditions:** order acceptance, price errors, availability, cancellation, payment, and complaint process, worded consistently with law.
- **Delivery policy:** areas served, dispatch estimates, delivery fees, failed delivery, COD terms, tracking limits.
- **Returns/refunds/exchanges/warranty:** clear eligibility, process, costs, timelines and exceptions; no unconditional guarantee unless approved and supportable.

- **Complaints/support:** usable channel, confirmation that a complaint was received, and a realistic response process without invented service-level promises.
- **Invoice/receipt:** required details based on seller status and tax advice; never fabricate a tax or registration number.

9.4 Tax data design rules

Keep separate fields/records for:

- product price and discount;
- store delivery fee;
- courier's actual charge/cost;
- payment provider fee;
- statutory sales tax, only where applicable and confirmed;
- income-tax withholding/collection records;
- final amount charged/collected and any refund.

Do not display withholding as if it were necessarily a sales tax charged to the customer. Do not decide tax treatment solely from product category or customer address without a verified rule.

10. Payments, courier, and WhatsApp research

10.1 Raast / advance payment

Verified official fact: SBP says merchants can accept Raast payments by enabling services through their bank or payment service provider. SBP also publishes Raast P2M information and participating providers. This does not prove that this seller's account is eligible for a particular API, QR mode, payment confirmation, fee, or settlement flow.

Sources:

- https://www.sbp.org.pk/our-subsidiaries/raast/join-raast
- https://www.sbp.org.pk/our-subsidiaries/raast/raast-person-to-merchant
- https://www.sbp.org.pk/circulars/pspod-circular-letter-no-01-of-2024

Recommended practical starting point: If bank-supported merchant Raast is available, consider manual/QR transfer with independent confirmation before paid status. Otherwise compare a properly licensed provider that can onboard this seller. Do not integrate until eligibility, fee schedule, API/webhook documentation, settlement, refund process, and test/sandbox access are confirmed. Do not promise "instant automatic confirmation" unless tested.

10.2 Courier

No courier has been selected or verified. Compare:

- coverage from the actual dispatch location;
- COD availability and onboarding requirements;
- COD commission, shipping fee, return-to-origin and reverse logistics costs;
- settlement frequency and deductions;
- pickup reliability, tracking, proof of delivery, claims, lost/damaged parcel process;
- official API documentation and production credentials;
- support and contract terms.

Fallback: manual dispatch and tracking updates by an authorized admin. Do not invent courier coverage, delivery estimates, fees, or tracking links.

10.3 WhatsApp

Recommendation: a click-to-chat link at launch. Automated WhatsApp Business Platform messaging should be deferred until account eligibility, applicable terms, customer opt-in, templates, pricing, and operational ownership are verified. A click-to-chat link is not permission to send unsolicited automated marketing. Official terms index: https://www.whatsapp.com/legal/

11. Cloudflare pricing, limits, and cost controls

Official documentation checked 25 October 2026. Prices/limits can change; recheck before implementation and before launch. The account's current plan/usage is unknown. No costs are incurred or resources created by this blueprint.

11.1 Pages

Current Pages limits documentation states:

- Free: 500 builds/month, one build at a time.
- Free: up to 20,000 files per site.
- Individual static asset maximum: 25 MiB.
- Pages Functions requests count against the Workers plan quota.

Source: https://developers.cloudflare.com/pages/platform/limits/

11.2 Pages Functions / Workers

Current documentation says Pages Functions are billed as Workers. The Free plan quota is 100,000 requests/day shared with other Workers usage; the daily limit resets at midnight UTC. Workers Paid currently lists a minimum charge of US$5/month, with included request/CPU allowances and charges beyond those allowances. D1 and other products have separate usage/pricing rules.

Sources:

- https://developers.cloudflare.com/pages/functions/pricing/
- https://developers.cloudflare.com/workers/platform/pricing/
- https://developers.cloudflare.com/workers/platform/limits/

Operational risk: On Free, exceeding a daily request or D1 row quota can cause requests/queries to fail until the reset. Do not assume "free" means no outage risk. Monitor usage, index queries, rate-limit abuse, and decide in advance what paid-plan trigger is acceptable.

11.3 D1

Current official D1 docs show:

- Free database size: 500 MB per database; total Free account storage: 5 GB.
- Free daily rows read: 5 million.
- Free daily rows written: 100,000.
- Free Time Travel retention: 7 days; Paid: 30 days.
- Paid plan includes monthly row-read/write allowances and storage allowance, with overage pricing as documented.

Sources:

- https://developers.cloudflare.com/d1/platform/limits/
- https://developers.cloudflare.com/d1/platform/pricing/

D1 Time Travel is useful but must not be the only recovery plan. Export backups and test restore procedures separately.

11.4 R2

Current official R2 pricing (last updated 1 October 2026 in the returned documentation) lists Standard storage at US$0.015/GB-month, Class A operations at US$4.50/million, Class B at US$0.36/million, and no internet egress charge. Standard free monthly allowance: 10 GB-month storage, 1 million Class A operations, and 10 million Class B operations. Infrequent Access has retrieval charges and different rates.

Source: https://developers.cloudflare.com/r2/pricing/

R2 buckets are private by default. Enabling a public bucket/custom domain makes its contents publicly accessible through that route. Product images may use a deliberate public delivery path; payment evidence must not. Sources:

- https://developers.cloudflare.com/r2/buckets/public-buckets/
- https://developers.cloudflare.com/r2/buckets/create-buckets/

11.5 Budget guardrails

Before implementation, owner should decide:

- acceptable monthly spend in USD/PKR;
- whether a minimum US$5/month Workers Paid plan is acceptable if required by traffic, reliability, or D1 usage;

- whether paid backups/other features are needed;
- what usage threshold should pause launch or trigger an owner approval.

No paid plan should be enabled automatically. Do not add Cloudflare products outside Pages, Functions, D1, and R2 without explicit approval; if a critical requirement appears to need another product, explain the gap and wait.

12. Mobile usability, accessibility, SEO, performance, and content standards

12.1 Mobile-first usability

- Design/test at narrow Android viewport widths, including 320–360 CSS pixels and larger common phone widths.
- Avoid horizontal scrolling, tiny tap targets, clipped dialogs, fixed elements that cover checkout controls, and forms that require precision.
- Use large enough touch targets, visible cart/checkout state, clear loading/empty/error/success messages, and preserve user input on recoverable errors.
- Test real mobile browser behavior, keyboard appearance, back navigation, slow connections, and orientation changes where relevant.
- Click-by-click instructions must match verified UI. If the user's screen differs, stop and ask what they see instead of inventing a path.

12.2 Accessibility

Target WCAG 2.2 AA as a project quality goal; this is a target, not a claim of certification or a determination of legal applicability.

- Semantic landmarks, heading hierarchy, labels, accessible names, alt text, keyboard navigation, visible focus, sufficient contrast, reduced-motion support.
- Forms identify errors in text and associate messages with the fields.
- Dialogs trap/return focus correctly; menus and accordions expose state to assistive technology.
- Status changes are announced appropriately; colour is not the only indicator.
- Product imagery has meaningful alt text; decorative images use empty alt.
- Test keyboard-only use and at least one screen-reader workflow on a practical available device/browser.

12.3 SEO

- Unique product/category titles and descriptions; canonical URLs; clean, stable product slugs.
- Correct `robots.txt` and sitemap behavior; exclude admin, checkout, account, private order lookup, and sensitive routes from indexing.
- Use structured product data only when values (price, currency, availability, image) are accurate and reflect the visible page.
- Do not publish draft/unavailable products as available; avoid duplicate thin category/product pages.
- Social metadata must not invent discounts, review ratings, stock, or product claims.
- Product pages must still provide meaningful information if JS/API fails, or clearly communicate the error without indexing a misleading empty shell. Final SEO implementation depends on the chosen Pages routing strategy.

12.4 Performance

- Optimize/compress responsive product images; prefer modern formats when broadly supported and preserve adequate image quality.
- Avoid unnecessary libraries, blocking scripts, oversized bundles, layout shifts, and large unoptimized hero images.
- Set image dimensions/aspect ratios, lazy-load below-the-fold images, and prioritize only the actual main visual.
- Cache public static assets; use short/no-store caching for private API responses.
- Measure actual performance on mobile using available tools; do not claim a Lighthouse/Core Web Vitals score without a recorded test.
- Consider that Pages site assets have a 25 MiB individual limit; use R2 for product images as designed.

12.5 Honest copy / CRO / trust

- State what ZEE COLLECTION sells, the genuine benefit, price, delivery terms, payment method, and next action clearly.
- Product pages should specify accurate materials, dimensions, functions, compatibility, included accessories, care, stock, and warranty only where verified.
- Do not fabricate reviews, ratings, testimonials, sales numbers, scarcity, discounts, guarantees, certifications, official affiliation, or delivery promises.
- Do not copy a competitor's protected text, imagery, or distinctive design. Inspiration is not permission to copy.
- Keep total price and separate charges clear before order submission.
- Support should confirm receipt of complaints and provide a realistic process without invented response-time promises.

13. Backup, recovery, logging, and incident response

13.1 Backup plan

Before production data exists:

- document D1 export/backup and restore procedure;
- document R2 object inventory/export strategy and metadata consistency;
- keep migrations and schema versions in GitLab;
- define who can access backups and how sensitive exports are protected;
- test restore into a separate non-production resource before relying on it;
- record recovery point objective (maximum tolerable data loss) and recovery time objective (maximum tolerable downtime) after owner input.

Cloudflare’s built-in recovery features do not replace a tested independent backup. Do not assume backup exists until an export is made and restored successfully.

13.2 Rollback

- Every risky schema/app change has a pre-change backup or documented rollback route.
- Deploy to preview/staging first where the actual Pages setup supports it.
- Verify deployment URL, health endpoint, catalogue, admin access, checkout/order flow, and error logs.
- Keep the prior known-good deployment available for rollback.
- Do not roll back a schema change blindly if new data has already been written; define forward-fix/restore decision first.

13.3 Logging and incident response

- Log request IDs, action, actor, entity, result, and timestamp for important admin/payment/order actions.
- Avoid passwords, tokens, full payment evidence, and unnecessary PII in logs.
- Document steps to disable admin access, rotate secrets, revoke sessions, investigate suspicious actions, notify affected parties where required, and restore data.
- Verify current legal/contractual breach-response obligations with qualified counsel; do not invent a notification deadline.

14. Testing strategy and evidence

Testing is not complete just because a page loads. Every test record should include environment, steps, expected result, actual result, status, date, and evidence (redacting private data).

14.1 Unit/logic checks

- Money arithmetic, rounding, discounts, delivery/tax calculation rules.
- State-transition validity.
- Input validation and field limits.
- Slug/SKU uniqueness.
- Guest token generation and hash/verification.
- Idempotency behavior.

14.2 Integration checks

- Public catalogue reads only published D1 records.
- Admin create/edit/archive/publish persists to D1.
- R2 image upload, D1 metadata association, display, replacement, and cleanup.
- Orders and item snapshots persist correctly.
- Stock decreases/reserves safely and is restored on approved cancellation paths.
- Payment status transitions and audit trail.
- Private evidence cannot be fetched anonymously.
- Admin session login/logout/expiry/revocation.

- API errors are consistent and safe.

14.3 Security checks

- Unauthenticated admin API calls fail.
- Non-admin sessions cannot use admin endpoints.
- User A cannot access User B's order.
- Guest lookup tokens cannot be guessed/enumerated; order number alone does not grant access.
- Client price/stock/status tampering is ignored/rejected.
- Duplicate order/payment requests are idempotent.
- Simultaneous purchases do not oversell.
- SQL injection/XSS/CSRF/CORS/upload abuse tests.
- Rate-limit tests, including persistence across requests.
- No secrets/hashes/private keys in public JS, responses, logs, or repository.

14.4 UX/accessibility/performance checks

- Narrow mobile layout and touch use.
- Keyboard-only and screen-reader paths.
- Focus, contrast, labels, validation and reduced motion.
- Slow/offline/failed API behavior.
- Product SEO metadata and robots/canonical/sitemap behavior.
- Image performance, caching, and no layout shifts.
- Test on an Android browser and record actual outcomes.

14.5 Operational checks

- D1 backup and successful restore.
- R2 object/metadata reconciliation.
- Deployment rollback rehearsal.
- Quota/cost review.
- Payment provider sandbox/controlled test and refund/reconciliation test if an integration is chosen.
- Courier manual/API workflow test.
- Policy review and support/complaint process test.

Never report a test as passed unless it was actually run. Tests not run must be listed with the reason and blocker.

15. Phases, deliverables, and acceptance gates

Phase 0 — Discovery and blueprint (current phase)

Tasks

- Compare all source documents; identify conflicts and unsupported assumptions.
- Verify official legal/tax/payment/provider/Cloudflare sources and date-check them.
- Confirm architecture, proposed schema/API, threat model, policies, costs, risk register, and open decisions.
- Verify domain/repository/current resource state only through authorized, safe inspection; do not change it.
- Deliver `BLUEPRINT.md` for owner review.

Deliverables

- Corrected blueprint, source register, open decisions, proposed acceptance criteria, test plan.
- No app code, no project-file creation in the project, no migrations, no resource creation, no configuration changes, no deployment.

Acceptance gate

- Owner reviews and explicitly approves the blueprint and resolves or accepts the listed blockers.

- Stop here until approval.

Phase 1 — Storefront foundation

Scope

- Semantic HTML, CSS design tokens/components, vanilla JS, responsive navigation, watch-first landing/catalogue UI, product-card/detail shell, loading/empty/error/offline states, accessible interactions, and clearly defined API integration boundaries.
- No false claim that catalogue/checkout is live before endpoints exist.
- Do not add admin/payment features prematurely.

Deliverables

- Complete files as downloadable artifacts; full ZIP of all project files where practical.
- Mobile screenshots/test notes if actually captured; test report; updated changelog and project docs.

Acceptance criteria

- Mobile layout works at agreed viewport widths without horizontal overflow.
- Keyboard focus, labels, contrast, reduced motion, and error states meet agreed checks.
- No console errors in tested flows; API failures are handled clearly.
- HTML/CSS/JS only; no external framework or persistence service.
- No secrets or fabricated content.
- Report exact files changed and tests run.

Gate: Stop and wait until owner says Phase 1 is completely done/approved.

Phase 2 — Admin and catalogue persistence

Scope

- Review/approve D1 migration; admin authentication; products/categories/variants/images CRUD; publish controls; R2 upload and cleanup.

Acceptance criteria

- D1 persistence verified across sessions; public API exposes only published data.
- Unauthorized access fails; admin roles checked server-side.
- Upload type/size/permission checks work; payment evidence stays private.
- Product/variant/stock/image consistency tested; audit records exist.
- Migration/backup/rollback documented and tested.

Gate: Stop and wait for owner approval.

Phase 3 — Checkout and order creation

Scope

- Cart/Buy Now, delivery/contact form, guest order creation and secure lookup; accounts only if approved.
- Server-side totals, availability, idempotency, inventory transaction, order snapshots.

Acceptance criteria

- Client price tampering rejected/ignored.
- No overselling in concurrent final-stock test.
- Double-submit/retry does not create duplicate orders.
- Customer/guest cannot access another order.
- Checkout displays a clear breakdown of product subtotal, delivery fee, applicable tax if verified, and amount due.
- Data collection and retention are documented.

Gate: Stop and wait for owner approval.

Phase 4 — Payments and order operations

Scope

- Integrate only an approved, eligible method; manual verification fallback; separate payment/order states; refund record/flow; settings and audit trail.

Acceptance criteria

- Provider/webhook authentication verified where applicable.
- Duplicate/replayed events do not double-apply.
- Amount, currency, order, and provider transaction are reconciled.
- Evidence never auto-approves payment.
- Refund/decline actions are permissioned and audited.
- Failure and reconciliation paths tested.

Gate: Stop and wait for owner approval.

Phase 5 — Shipping, returns, and support

Scope

- Manual or verified courier integration, tracking, delivery failure/return-to-origin, return/refund/exchange workflows, complaint intake, approved notifications.

Acceptance criteria

- Every state transition is validated/audited.
- No invented tracking, coverage, or delivery promise.
- Customer-facing policy matches actual operations and legal review.
- Failed courier integration has a usable manual fallback.
- Support complaints receive a clear receipt confirmation.

Gate: Stop and wait for owner approval.

Phase 6 — Hardening and launch

Scope

- Full security/accessibility/performance/SEO checks, backup/restore and rollback rehearsal, legal/policy review, domain and deployment setup, monitoring, production readiness.

Acceptance criteria

- No unresolved critical/high risks without explicit owner acceptance and mitigation.
- Production secrets are configured privately and tested without disclosure.
- D1/R2 recovery procedures have evidence.
- Payment/courier operations and support process have been rehearsed.
- Current tax/legal advice and required policies are confirmed.
- Domain ownership/DNS is verified and change plan includes rollback.
- Deployment smoke tests pass; monitoring/quota/cost plan exists.
- Owner explicitly authorizes launch.

Gate: Do not launch until owner approves.

16. Risk register

Risk	Severity	Mitigation / decision
Admin account/session compromised	Critical	Secure sessions, rate limits, role checks, audit, recovery, optional MFA, least privilege
Customer/order/evidence exposure	Critical	Default-deny authorization, private R2 evidence, ownership tests, data minimization
Price/stock tampering or overselling	High	Server-side calculation, transactions/conditional stock updates, idempotency, concurrency tests
Wrong tax/registration assumption	High	Current official sources, seller-specific tax advice, no hard-coded unverified rate
Payment evidence fraud/replay	High	Independent verification, signed webhook checks, event uniqueness, audit
Domain/DNS accidentally changed	High	Inspect first, document current state, owner approval, rollback plan
Provider/merchant ineligible	High	Confirm onboarding, fees, API access and settlement before integration; manual fallback
R2 public exposure of private evidence	Critical	Private bucket, authorization proxy, public-access verification tests

| Upload abuse/malicious content | High | Content validation/re-encoding, limits, generated keys, private evidence |

| D1/Functions quota exhaustion | High | Usage monitoring, indexed queries, rate limits, budget threshold and upgrade decision |

| Backup unusable | High | Independent export and restore rehearsal |

| Courier API unavailable | Medium | Manual dispatch/tracking workflow |

| Legal/customer policy inaccurate | High | Owner-approved policies and qualified legal/tax review |

| Mobile/accessibility failures | Medium/High | Mobile-first acceptance tests, keyboard/screen-reader checks |

| Scope expands before core flows work | Medium | Phase gates and written approval |

| Static page implies live feature that does not exist | Medium | Clear UI boundaries and API-failure states |

17. Unresolved decisions for the owner

Only decisions that materially affect design, legal review, costs, or launch should block the next step.

1. **What is the launch catalogue?**

Options: watches only; watches plus shoes/fashion; all categories at launch.

Recommendation: watches only; add rings/glasses after the first launch works.

2. **What is the seller's business identity and dispatch origin?**

Needed to check disclosures, tax, consumer law, courier availability and fees.

Options may include an individual/sole proprietor or a registered entity; choose the truthful current status, not a status you hope to obtain later.

Recommendation: provide the actual legal/trading name and city/area privately in your own records and use a qualified Pakistani professional to confirm required public disclosures. Do not post sensitive identity documents in chat.

3. **Where will you sell and in what currency?**

Options: selected cities, Pakistan-wide, or international; PKR or other supported currency.

Recommendation: Pakistan-only and PKR at launch, if consistent with your actual operations, to reduce shipping/tax complexity.

4. **Do you own `zeecollection.tech`, and is it already connected to Cloudflare?**

Options: owned and managed; owned elsewhere; not owned/uncertain.

Recommendation: verify registrar and current DNS before buying anything or changing nameservers.

5. **Which payment method can your seller account actually use?**

Options: COD, bank-supported Raast/QR/manual transfer, or a verified payment provider.

Recommendation: COD only if courier onboarding is confirmed, plus one advance method only after eligibility, fees, verification and refund handling are verified. Do not assume a Raast API is available.

6. **Which courier can collect from your dispatch location?**

Options: courier with COD/API, courier with manual workflow, or self-arranged delivery.

Recommendation: choose based on actual pickup area, COD settlement, RTO costs, coverage, tracking, and support—not brand popularity alone.

7. **Guest checkout or customer accounts?**

Options: guest checkout only initially; accounts at launch; guest plus optional accounts.

Recommendation: guest checkout with secure order lookup first, unless you need account features for a concrete business reason.

8. **What return/refund/warranty and delivery policy can you actually honor?**

Options depend on product quality, supplier, shipping, and legal advice.

Recommendation: use a clear, realistic policy confirmed with a Pakistani legal professional; do not promise an unconditional guarantee until approved.

9. **What monthly operating budget is acceptable?**

Options: stay within verified Free limits; approve a minimum paid Workers plan if needed; set a hard monthly cap.

Recommendation: set a hard cap and a threshold at which implementation pauses for approval. Current Workers Paid documentation lists a US$5/month minimum, but account-specific costs and future usage must be checked.

10. **Do you want automated WhatsApp messages or just click-to-chat?**

Options: click-to-chat; automated Business Platform messaging.

Recommendation: click-to-chat at launch; defer automated messaging until eligibility, consent, templates, pricing and ownership are confirmed.

11. **Should analytics/ad pixels be included?**

Options: none at launch; privacy-reviewed first-party analytics; third-party tracking.

Recommendation: no advertising pixels or third-party trackers at launch until purpose, privacy, consent, and data-sharing implications are reviewed.

12. **What is the minimum acceptable recovery target?**

Options: tolerate up to a day of lost changes; require a tighter recovery point; decide after estimating order volume.

Recommendation: at minimum, make regular independent D1 exports and test restore before accepting real orders; choose the exact frequency based on order volume and operational burden.

18. Owner approval checklist

Before implementation begins, owner should confirm or explicitly accept as unresolved:

- [ ] Watches-only launch scope.
- [ ] Seller identity/legal form and dispatch origin are known for professional review.
- [ ] Sales geography and currency.
- [ ] Domain ownership/DNS verification plan.
- [ ] Payment approach and fallback.
- [ ] Courier and manual fallback.
- [ ] Guest/account approach.
- [ ] Draft delivery/returns/refund/warranty policies.
- [ ] Privacy/retention and analytics decisions.
- [ ] Monthly cost cap and recovery expectations.
- [ ] This architecture and proposed schema/API plan.
- [ ] Phase 1 scope and acceptance gate.

19. Source register (official sources checked 25 October 2026)

Pakistan law, tax, and payments

- National Assembly — Prevention of Electronic Crimes (Amendment) Act 2025: https://na.gov.pk/uploads/documents/679b243193585_457.pdf
- NCCIA — laws listing: https://www.nccia.gov.pk/laws.php
- Pakistan Code — Electronic Transactions Ordinance, 2002: https://pakistancode.gov.pk/pdffiles/administratordbc98dd49f2df3b1d07bb986dcceb9a3.pdf
- Ministry of Commerce — e-Commerce Policy: https://www.commerce.gov.pk/ecommerce-policy-2019/
- FBR — Income Tax Circular No. 01 of 2025–26: https://download1.fbr.gov.pk/Docs/2025841183918948CircularNo01of2025-26IncomeTax.pdf
- FBR — Sales Tax/Federal Excise Circular No. 02 of 2025–26: https://download1.fbr.gov.pk/Docs/202584118361586CircularNO02of2025-26SalesTax%26FederalExcise.pdf
- FBR — SRO 1429(I)/2025: https://download1.fbr.gov.pk/SROs/2025858833282SRO1429DATED04-08-2025.pdf
- FBR — Income Tax Ordinance amended 20 February 2026: https://download1.fbr.gov.pk/Docs/2026226162211364IncomeTaxOrdinance2001-Amended-20.02.2026.pdf
- MOITT — legislation listing: https://www.moitt.gov.pk/Legislations
- State Bank of Pakistan — Raast merchant joining: https://www.sbp.org.pk/our-subsidiaries/raast/join-raast
- State Bank of Pakistan — Raast P2M: https://www.sbp.org.pk/our-subsidiaries/raast/raast-person-to-merchant
- State Bank of Pakistan — Raast P2M acceptance service circular: https://www.sbp.org.pk/circulars/pspod-circular-letter-no-01-of-2024
- WhatsApp — legal terms index: https://www.whatsapp.com/legal/

Cloudflare

- Pages limits: https://developers.cloudflare.com/pages/platform/limits/
- Pages Functions pricing: https://developers.cloudflare.com/pages/functions/pricing/
- Workers limits: https://developers.cloudflare.com/workers/platform/limits/
- Workers pricing: https://developers.cloudflare.com/workers/platform/pricing/
- D1 limits: https://developers.cloudflare.com/d1/platform/limits/
- D1 pricing: https://developers.cloudflare.com/d1/platform/pricing/

- R2 pricing: https://developers.cloudflare.com/r2/pricing/
- R2 public buckets: https://developers.cloudflare.com/r2/buckets/public-buckets/
- R2 bucket creation/default access notes: https://developers.cloudflare.com/r2/buckets/create-buckets/

Source limitations

- Current official-source checks support the specific facts stated above, not a blanket claim that every applicable legal, tax, provider, or product requirement has been conclusively resolved.
- Consumer-protection applicability and seller-specific tax treatment remain open.
- Payment-provider/courier eligibility, pricing, account access, and API readiness remain unverified because no provider/account was selected and tested.
- Domain ownership, Cloudflare configuration, GitLab state, current local files, and current deployment were not inspected or changed during this review.
- Recheck sources at the moment implementation begins; legal, tax, pricing, account limits, and provider terms can change.

20. Final status

BLUEPRINT STATUS: awaiting owner review.

This corrected blueprint is a planning deliverable only. The project remains paused. No code, database migrations, D1/R2 resources, Cloudflare settings, DNS, GitLab repository changes, payment/courier integrations, or deployment changes have been made as part of this review.

After reviewing this document, tell me which decisions you want to confirm or correct. I will revise the blueprint only. Implementation starts only after you explicitly authorize it.